

**«Санкт-Петербургский государственный электротехнический университет
«ЛЭТИ» им. В.И.Ульянова (Ленина)»
(СПбГЭТУ «ЛЭТИ»)**

ОТЧЁТ ПО УЧЕБНОЙ ПРАКТИКЕ

Тема: ГЕНЕТИЧЕСКИЕ АЛГОРИТМЫ

Студенты гр.
4345

М.А. Гавриш
А.Д. Шувалов
А. Трушкин

Руководитель

Т.Р. Жангиров

Санкт-Петербург
2026

ЗАДАНИЕ НА УЧЕБНУЮ ПРАКТИКУ

Студенты: М.А. Гавриш, А.Д. Шувалов, А. Трушкин

Группа: 4345

Тема практики: Генетические алгоритмы

Задание на практику:

Разработать программу с графическим пользовательским интерфейсом для решения задачи аппроксимации полиномиальной функции $f(x)$ степени не выше 8 ступенчатой функцией $g(x)$ с заданным количеством ступеней на интервале $[l, r]$.

Сроки прохождения практики: 06.07.2026 – 18.06.2026

Дата сдачи отчета: 17.06.2026

Дата защиты отчета: 17.06.2026

Студент	_____	М.А. Гавриш
Студент	_____	А.Д. Шувалов
Студент	_____	А. Трушкин
Руководитель	_____	Т.Р. Жангиров

АННОТАЦИЯ

Краткое изложение содержания отчёта.

SUMMARY

Аннотация на английском языке.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	5
1 ПОСТАНОВКА ЗАДАЧИ	6
1.1 Предметная область	6
1.2 Задачи практики	6
2 РЕЗУЛЬТАТЫ РАБОТЫ.....	8
2.1. Изучение теоретических основ.....	8
2.2. Изучение методов аппроксимации.....	8
2.3. Составление плана	8
2.3.1. Представление данных.....	8
2.3.2. Функция качества	8
2.3.3. Общий алгоритм	8
2.3.4. Прототип графического интерфейса будущей программы.....	9
2.3.5. Модуль интеграции и работы с данными	9
2.4. Генетический алгоритм	10
2.4.1. Функция приспособленности	11
2.4.2. Основные составляющие алгоритма	11
2.5. GUI.....	12
2.5.1. Главное окно	12
2.5.2. Виджеты и объекты	13
2.6. Управление данными и конфигурацией алгоритма	13
2.6.1. Класс GAConfig	14
2.6.2. Класс DataManager	14
2.6.3. Контроллер приложения	15
3 ИСПОЛЬЗУЕМЫЕ ТЕХНОЛОГИИ	17
4 ОТЗЫВ РУКОВОДИТЕЛЯ	18
ЗАКЛЮЧЕНИЕ	19
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	20
ПРИЛОЖЕНИЕ А	21

ВВЕДЕНИЕ

Генетические алгоритмы представляют собой эвристические методы оптимизации, основанные на принципах естественного отбора и генетики. Данные алгоритмы находят широкое применение в задачах, где традиционные методы оптимизации оказываются неэффективными или неприменимыми ввиду высокой размерности пространства решений, нелинейности целевой функции или отсутствия аналитического решения.

Предметной областью данной практики является разработка программы для численной аппроксимации функций с использованием эволюционных вычислений. Использование генетических алгоритмов для решения данной задачи позволяет находить приемлемые решения в условиях оптимизации параметров аппроксимирующей функции.

Целью практики является разработка программы с графическим интерфейсом, реализующего генетический алгоритм для минимизации расстояния между полиномиальной функцией и ступенчатой функцией заданной структуры.

Для достижения поставленной цели необходимо решить следующие задачи: изучить теоретические основы генетических алгоритмов и методов аппроксимации функций, разработать структуру данных для представления полиномов и ступенчатых функций, реализовать операторы генетического алгоритма (селекцию, скрещивание, мутацию), создать функцию качества для оценки приближения, разработать графический интерфейс с возможностью визуализации процесса поиска решения, обеспечить возможность настройки параметров алгоритма и ввода данных.

1 ПОСТАНОВКА ЗАДАЧИ

1.1 Предметная область

Задача приближения полинома ступенчатой функцией формулируется как задача минимизации расстояния между функциями. Параметры оптимизации — высоты ступеней ступенчатой функции при фиксированных границах разбиения интервала. Данная задача является задачей непрерывной оптимизации.

Практическая значимость задачи обусловлена необходимостью разработки инструментов для визуального анализа процесса аппроксимации, что важно для исследования свойств генетических алгоритмов. Разработка программы с пошаговой визуализацией позволяет изучить разницу между популяциями.

1.2 Задачи практики

1. Изучить теоретические основы генетических алгоритмов (селекция, скрещивание, мутация).
2. Изучить методы аппроксимации функций, метрики расстояния между функциями.
3. Разработать структуру данных для представления полиномиальной функции с заданным интервалом определения.
4. Разработать структуру данных для представления ступенчатой функции с фиксированными границами ступеней.
5. Реализовать функцию качества для оценки степени приближения ступенчатой функции к полиному на заданном интервале.
6. Реализовать основные операторы ГА: инициализацию популяции, селекцию, скрещивание, мутацию и формирование нового поколения.
7. Разработать механизм сохранения истории поколений для обеспечения возможности возврата к предыдущим состояниям алгоритма.
8. Спроектировать и реализовать графический интерфейс с возможностью управления параметрами алгоритма.

9. Реализовать возможность ввода данных разными способами: из файла и путём случайной генерации.
10. Обеспечить пошаговую визуализацию процесса эволюции и перехода к конечному решению.

2 РЕЗУЛЬТАТЫ РАБОТЫ

Разделение ролей и ответственность:

1. Шувалов Алексей – разработка генеративного алгоритма.
2. Трушкин Александр – разработка графического интерфейса.
3. Гавриш Матвей – разработка связи между ГА и GUI.

2.1. Изучение теоретических основ

Были изучены теоретические основы генетических алгоритмов: основные операторы и свойства.

2.2. Изучение методов аппроксимации

Был изучен метод аппроксимации с помощью генетического алгоритма

2.3. Составление плана

Был составлен план по решению задачи:

2.3.1. Представление данных

Целевая функция $f(x)$: используется массив целых чисел для хранения коэффициентов функции, где индекс элемента соответствует степени x .

Ген: вещественное число, равное высоте одной ступени.

Генотип: одномерный массив вещественных чисел, длина которого равна количеству ступеней. Каждый генотип является полноценной ступенчатой функцией.

Популяция: двумерный массив, состоящий из N генотипов.

2.3.2. Функция качества

Расчёт ошибки: $Error = \frac{1}{N} \sum_{i=0}^N |f(x_i) - g(x_i)|$

Приспособленность: $Fitness = \frac{1}{1+Error}$

2.3.3. Общий алгоритм

1. Ввод данных: пользователь программы вводит коэффициенты полиномиальной функции, интервал, количество ступеней и параметры генетического алгоритма (размер популяции, число поколений и шанс мутации). Доступен ручной ввод, случайная генерация данных, чтение из файла.

2. Генерация начальной популяции

3. Цикл эволюции

- Оценка приспособленности для каждой особи в популяции.
- Сохранение лучшей особи.
- До тех пор, пока не заполнится новая популяция:
 - Турнир для выявления родителей.
 - Скрещивание.
 - Мутация потомков.
 - Добавление потомков в новую популяцию.
 - Замена старой популяции на новую.

4. Вывод результата: лучший найденный генотип содержит искомые высоты ступеней.

2.3.4. Прототип графического интерфейса будущей программы

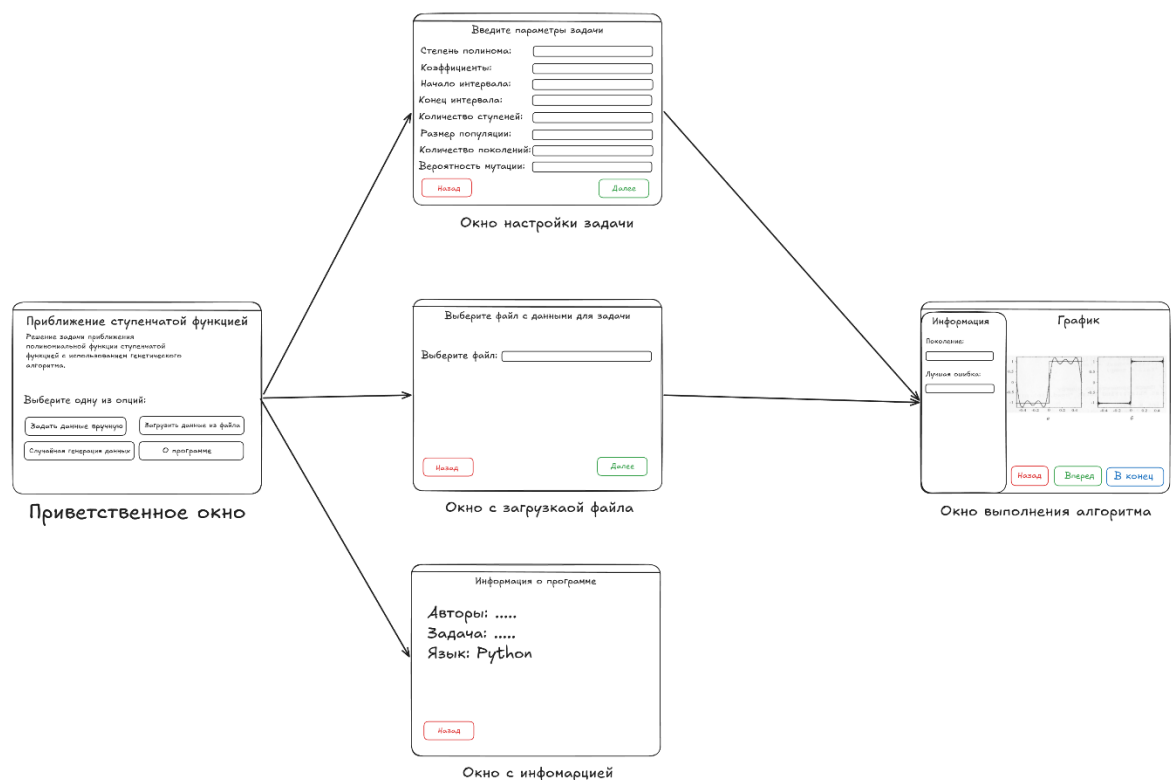


Рисунок 1 – Прототип графического интерфейса

2.3.5. Модуль интеграции и работы с данными

Данный модуль выполняет роль контроллера в архитектуре приложения. Он обеспечивает связь между графическим интерфейсом (gui-prototype) и вычислительным ядром (backend-prototype). Основные задачи модуля:

- Валидация и нормализация входных данных
- Чтение/запись параметров задачи из файлов
- Случайная генерация тестовых полиномов
- Управление циклом алгоритма (пошаговый запуск, переход вперёд/назад, быстрый пропуск)

- Хранение истории состояний для визуализации

Логика случайной генерации данных:

- Для режима случайная генерация будет реализован алгоритм создания валидных тестовых данных:

- Степень полинома: случайно выбирается от 1 до 8.
- Коэффициенты: генерируются случайные целые числа в диапазоне $[-5, 5]$.
- Интервал: случайно выбирается в диапазоне $[-10, 10]$ при условии $l < r$.
- Количество ступеней: случайно выбирается от 10 до 35.

Для реализации требования о возможности вернуться на несколько шагов назад модуль хранит полную историю эволюции. При каждом шаге алгоритма бэкенд возвращает новую популяцию, и контроллер сохраняет её в `self.generations`. При нажатии кнопок «Назад»/«Вперед» в GUI контроллер не пересчитывает алгоритм, а просто меняет `current_step` и отдает GUI сохраненный снимок для отрисовки.

2.4. Генетический алгоритм

Генетический алгоритм предназначен для приближения заданной полиномиальной функции $f(x)$ ступенчатой функцией $g(x)$ на заданном интервале $[l, r]$. Цель алгоритма – подбор параметров ступеней, при которых сумма разностей между высотами графиков функций будет минимальной. Для хранения коэффициентов исходной функции используется массив целых чисел.

Генотип хранится в виде массива вещественных чисел, длина которого равняется количеству ступеней. Каждый ген в генотипе представляет собой

высоту соответствующей ступени. Стартовые значения генов генерируются случайно в пределах от минимального до максимального значения исходной полиномиальной функции.

2.4.1. Функция приспособленности

Оценка генотипа происходит путём суммирования разниц между значениями функций в определенных точках. Данные точки определяются путём разбиения отрезка $[l, r]$ на равные отрезки.

Значение приспособленности вычисляется путем инверсии ошибки. Приспособленность будет стремиться к единице, так как при совпадении графиков ошибка будет равняться нулю.

Для имитации эволюции в данном алгоритме используются: турнирный отбор для трёх генотипов, одноточечное скрещивание и мутация с заданными вероятностями.

2.4.2. Основные составляющие алгоритма

Класс `Polynom` – предназначен для вычисления значения полиномиальной функции в точке и для вычисления её верхней и нижней границ.

Класс `StepFunc` – математическая модель ступенчатой функции. Позволяет вычислить значение функции в точке.

Метод `init_population` – создаёт начальную популяцию случайных особей, распределяя значения генов в вычисленных границах.

Метод `calc_fitness` – вычисляет значения приспособленности для разных генотипов.

Метод `tournament_selection` – выбирает лучший генотип из трёх кандидатов в рамках турнирного отбора.

Метод `crossover` – с заданной вероятностью проводит одноточечное скрещивание двух генотипов для создания новых потомков.

Метод `mutation` – с заданной вероятностью проводит мутацию одного гена в генотипе в пределах 10%.

Метод `generation_step` – осуществляет один шаг эволюции. Сохраняет лучшую особь предыдущего поколения, отбирает родителей, скрещивает их, мутирует потомков и формирует новую популяцию.

Метод `save_generation_state` – сохраняет текущее поколение и его метрики на каждом шаге в список словарей.

Метод `go_to_generation` – переход к определенному поколению по индексу.

2.5. GUI

Для разработки графического интерфейса была выбрана экосистема Qt – промышленный стандарт для десктопных приложений.

- Библиотека GUI: PySide6.
- Инструмент визуального проектирования: Qt Designer – использовался для создания разметки, размещения элементов и управления их свойствами без написания интерфейса вручную.
- Утилита компиляции: `pyside6-uic` – применялась для автоматической конвертации визуального файла разметки `welcome.ui` формата XML в исполняемый Python-код `welcome_ui.py`.

2.5.1. Главное окно

Основой приложения является класс `MainWindow`.

Разрешение окна зафиксировано на 800×600 пикселей, что гарантирует стабильное отображение интерфейса без искажений на любых мониторах.

Многостраничность: реализована с помощью контейнера `QStackedWidget` (`self.ui.stackedWidget`). Интерфейс разделен на изолированные экраны, переключение между которыми происходит по индексам в коде `setCurrentIndex()`.

Архитектура страниц в проекте:

1. `welcomePage` (Индекс 0): приветственный экран. Содержит название программы, краткое описание и блок главных кнопок для выбора режима работы.

2. `startPage` (Индекс 1): страница ручного ввода параметров задачи. Именно здесь пользователь настраивает математические свойства для аппроксимации.

2.5.2. Виджеты и объекты

Внутри страниц были использованы и настроены следующие типы объектов PySide6:

- `QFrame` (`mainCard`, `mainCard_2`, `mainCard_3`): специальные контейнеры с белым фоном, внутри которых группируется весь контент страниц. Они отделяют рабочую зону от общего серого фона приложения.
- `QLabel`: использовались для всех текстовых блоков:
 - Заголовки и описания: `titleLabel`, `descriptionLabel`, `descriptionParamLabel`.
 - Подписи к полям ввода: `stepsLabel` (Количество ступеней), `beginningIntLabel` (Начало интервала) и др.
- `QPushButton`: Кнопки управления. Все кнопки имеют фиксированный минимальный размер 280×50 пикселей:
 - `startButton` – кнопка «Ввести данные вручную» на главном экране.
 - `aboutButton` – кнопка «О программе».
 - `backButton_1` – кнопка «Назад» для возврата на приветственный экран.
 - `nextButton_1` – кнопка «Вперед» для перехода по этапам настройки.
- `QSpinBox`: Поля для ввода числовых параметров.
- `QLineEdit` (`coefLineEdit`): Однострочное текстовое поле. Выбрано как оптимальное решение для ввода нескольких коэффициентов полинома строкой через пробел.

2.6. Управление данными и конфигурацией алгоритма

Для возможности взаимодействия с графическим интерфейсом был разработан модуль управления данными и конфигурацией. Данный модуль отвечает за генерацию задач, сохранение и загрузку параметров из файлов, валидацию входных данных, а также связывание графического интерфейса с вычислительным ядром генетического алгоритма.

2.6.1. Класс GAConfig

Класс GAConfig предназначен для хранения и валидации параметров генетического алгоритма. Все параметры вынесены в единый конфигурационный объект, что позволяет легко передавать настройки между модулями программы и изменять их через графический интерфейс.

Атрибуты класса:

- `population_size` — размер популяции
- `generations` — максимальное количество поколений
- `tournament_size` — количество участников турнирного отбора
- `mutation_chance` — вероятность мутации одного гена
- `crossover_rate` — вероятность скрещивания двух особей

Метод `validate` — производит проверку параметров алгоритма. Если пользователь задаёт значение, выходящее за допустимые границы, оно автоматически приводится к ближайшему допустимому значению. Это предотвращает некорректную работу алгоритма при ошибочном вводе.

2.6.2. Класс DataManager

Класс DataManager реализует функционал работы с входными и выходными данными программы. Он обеспечивает три способа получения параметров задачи: ручной ввод через графический интерфейс, загрузку из файла и случайную генерацию.

Метод `generate_random_polynom` — создаёт случайный полином заданной степени со случайными коэффициентами в заданном диапазоне.

Метод `generate_random_task` — формирует полный набор параметров задачи, включая случайный полином, интервал аппроксимации и случайное количество ступеней. Возвращает словарь с параметрами, готовый для передачи в генетический алгоритм.

Метод `save_to_file` — сохраняет параметры задачи и конфигурацию генетического алгоритма в файл формата JSON. Есть возможность сохранять только параметров задачи, так и совместное сохранение с настройками алгоритма.

Метод `load_from_file` — загружает параметры задачи из JSON-файла. Метод возвращает кортеж из словаря с параметрами задачи и объекта конфигурации.

Метод `validate_task_data` — метод валидации загруженных данных. Проверяет наличие всех обязательных полей и их значения. При обнаружении ошибок выбрасывает исключение `ValueError`.

2.6.3. Контроллер приложения

Класс `AppController` связывает графический интерфейс и вычислительные модули программы.

Метод `connect_signals` — устанавливает связь между элементами GUI и методами обработки событий.

Метод `on_manual_input` — обработчик кнопки ручного ввода данных. Запускает процесс перехода к окну ввода параметров полинома, интервала и количества ступеней.

Метод `on_load_file` — обработчик кнопки загрузки данных из файла. Открывает стандартный диалог выбора файла, вызывает метод загрузки `DataManager` и обрабатывает возможные ошибки. При успешной загрузке сохраняет параметры в контроллере для последующего использования алгоритмом.

Метод `on_random_generate` — обработчик кнопки случайной генерации данных. Вызывает метод генерации `DataManager` и сохраняет результат для последующей передачи в генетический алгоритм.

Метод `on_about` — обработчик кнопки «О программе».

Метод `run` — запускает главное окно приложения и передаёт управление циклу обработки событий графического интерфейса.

3 ИСПОЛЬЗУЕМЫЕ ТЕХНОЛОГИИ

В данной главе опишите технологии, которые использовались вами при выполнении задач в рамках практики.

Это могут быть языки программирования, списки конкретных библиотек / ПО и целей / мест их применения.

4 ОТЗЫВ РУКОВОДИТЕЛЯ

Если прохождение практики было по кафедральным темам достаточно добавить формулировку:

По результатам работы учебная практика оценена на <ваша_оценка>.

Если прохождение практики было по своей теме, то необходимо добавить скан отзыва руководителя с оценкой и подписью, скан должен быть в высоком качестве на всю страницу.

Для прохождения автоматической проверки для скана необходима подпись и ссылка в тексте, т.к. он считается рисунком.

Отзыв руководителя с оценкой (см. рис. X).

Отзыв
о прохождении учебной практики

<ФИО студента>, студент(ка) группы <номер группы>, второго курса бакалавриата Санкт-Петербургского электротехнического университета “ЛЭТИ” им В.И. Ульянова, проходил(а) учебную практику по теме “<тема практики>” с <дата начала> по <дата окончания>.

В течение практики обучающийся(аяся) <ФИО студента> выполнял <задание на практику>.

В результате практики обучающийся(аяся) <результат практики>. Получаемую работу обучающийся(аяся) <ФИО студента> выполнял <характеристика личных качеств практиканта>.

По результатам работы <ФИО студента> учебной практику оцениваю на <оценка: Отлично, Хорошо, Удовлетворительно, Неудовлетворительно>.

Подпись руководителя практики:

<Должность><дата> <подпись>/<ФИО>

Рисунок X — Отзыв руководителя

ЗАКЛЮЧЕНИЕ

Заключение содержит краткое описание результатов по каждой задаче, обозначенной в разделе «Постановка задачи», результаты работы в целом, согласно обозначенной цели и пр.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. The State of the Octoverse [Электронный ресурс]. URL: <https://octoverse.github.com> (дата обращения: 28.04.2021).
2. Бобкова, О.В., Давыдов, С.А., Ковалева, И.А., Плагиат как гражданское правонарушение / О.В. Бобкова, С.А. Давыдов, И.А. Ковалева // Патенты и лицензии. – 2016. – № 7. – С. 31-37.

ПРИЛОЖЕНИЕ А